Machine Learning (CE 40717)

Fall 2024

Ali Sharifi-Zarchi

CE Department
Sharif University of Technology

October 5, 2024

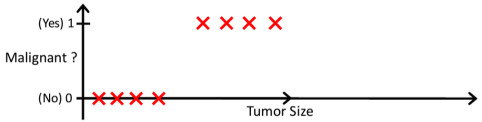
Classification problem

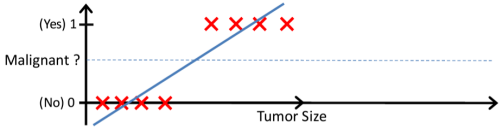
- **Classification (binary)**
  - Email: Spam / Not Spam?
  - Online Transactions: Fraudulent / Genuine?
  - Tumor: Malignant / Benign?

$$\mathbf{y} \in \{\mathbf{0}, \mathbf{1}\} \begin{cases} 0\text{: “Negative Class” (e.g., benign tumor)} \\ 1\text{: “Positive Class” (e.g., malignant tumor)} \end{cases}$$

Classification problem (cont.)

- Can we solve the problem using linear regression?



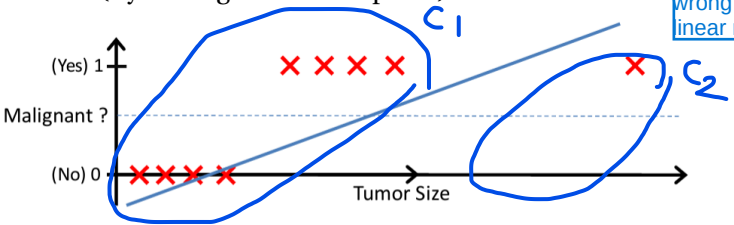


- We could fit a straight line and define a threshold at 0.5:

$$\textbf{If } h_\theta(x) \geq 0.5, \text{predict } y = 1$$
$$\textbf{If } h_\theta(x) < 0.5, \text{predict } y = 0$$

Classification problem (cont.)

- What about now? (By adding a new data point)



- Classification: $y = 0$ or $y = 1$
  - $h_\theta(x)$ can be > 1 or < 0
  - Another drawback of using linear regression for this problem
- What we need:

Logistic regression: $0 \leq h_\theta(x) \leq 1$

logistic regression is clasification

- We also show this function with other notations: $f(x; w) = \sigma(w^T x)$

Introduction

- Suppose we have a binary classification task (so $K = 2$).
- By observing age, gender, height, weight and BMI we try to distinguish if a person is overweight or not overweight.

Age	Gender	Height (cm)	Weight (kg)	BMI	Overweight
25	Male	175	80	25.3	0
30	Female	160	60	22.5	0
...					
35	Male	180	90	27.3	1

- We denote the features of a sample with vector x and the label with y.
- In logistic regression we try to find an $\sigma(w^T x)$ which predicts **posterior** probabilities $P(y = 1|x)$.

Introduction (cont.)

- $\sigma(w^T x)$: probability that $y = 1$ given x (parameterized by $\mathbf{w}$)

$$P(y = 1|x, \mathbf{w}) = \sigma(\mathbf{w}^T x)$$
$$P(y = 0|x, \mathbf{w}) = 1 - \sigma(\mathbf{w}^T x)$$

- We need to look for a function which gives us an output in the range [0, 1]. (like a probability).
- Let's denote this function with $\sigma(.)$ and call it the **activation function**.

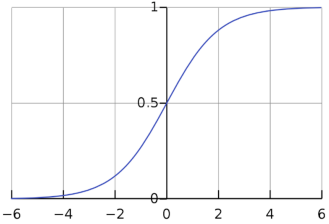
Introduction (cont.)

- Sigmoid (logistic) function.

normal distribution's probability function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

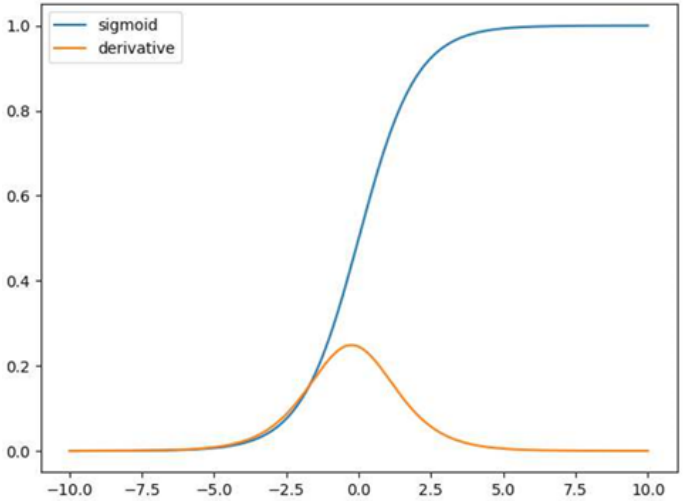
- A good candidate for activation function.
- It gives us a number between 0 and 1 **smoothly**.
- It is also **differentiable**



$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\left(1 - \sigma(z)\right)$$

$$\sigma(z) = \frac{1}{1+e^{-z}} \qquad 1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}}$$

Sigmoid function & its derivative



Introduction (cont.)

- The sigmoid function takes a number as input but we have:

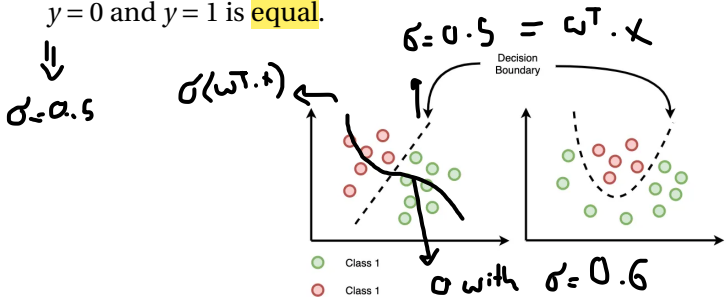
$$x = [x_0 = 1, x_1, \ldots, x_d]$$
$$w = [w_0, w_1, \ldots, w_d]$$

- So we can use the **dot product** of x and w.
- We have $0 \leq \sigma(\mathbf{w}^T x) \leq 1$. which is the estimated probability of $y = 1$ on input x.
- An Example : A basketball game (Win, Lose)
  - $\sigma(\mathbf{w}^T x) = 0.7$
  - In other terms 70 percent chance of winning the game.

$$\sigma = \frac{1}{1 + e^{-(w^T \cdot x)}}$$

Decision surface

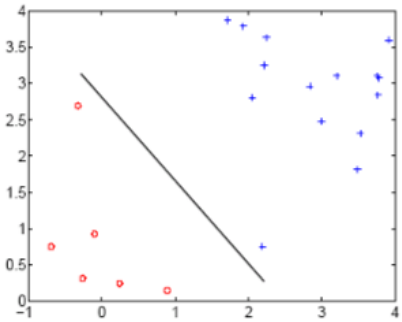
- Decision surface or decision boundary is the region of a problem space in which the output label of a classifier is ambiguous. (could be linear or non-linear)
- In binary classification it is where the probability of a sample belonging to each $y = 0$ and $y = 1$ is equal.



- Decision boundary hyperplane always has **one less dimension** than the feature space.

Decision surface (cont.)

- An example of linear decision boundaries:



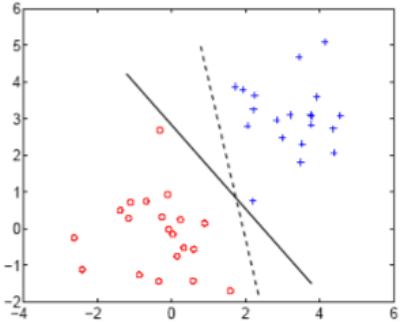


Figure adapted from Eric Xing, Machine Learning, CMU

Decision surface (cont.)

- Back to our logistic regression problem.
- Decision surface $\sigma(\mathbf{w}^T x) = \textbf{constant}$.

$$\sigma(\mathbf{w}^T x) = \frac{1}{1 + e^{-(\mathbf{w}^T x)}} = 0.5$$

- Decision surfaces are **linear functions** of x
  - if $\sigma(\mathbf{w}^T x) \geq 0.5$ then $\hat{y} = 1$, else $\hat{y} = 0$
  - Equivalently, if $\mathbf{w}^T x + w_0 \geq 0.5$ then decide $\hat{y} = 1$, else $\hat{y} = 0$

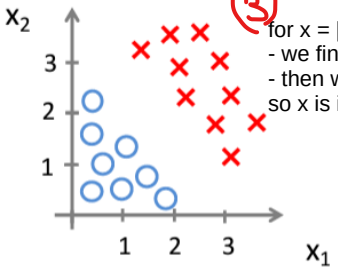
$z \sim \sigma$ چون $\sigma(z)$ صعودی است

$\hat{y}$ **is the predicted label**

Decision boundary example

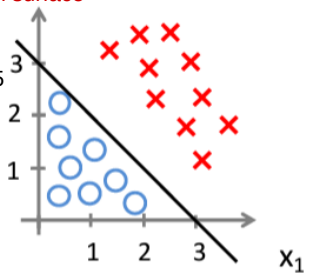
$$\sigma(\mathbf{w}^T x) = \sigma(w_0 + w_1 x_1 + w_2 x_2)$$

1. - w = [-3,1,1]
   so wTx = -3+x1+x2 decision surface



3. for x = [1,2,4]
   - we find : wTx: [-3,1,1][1,2,4] =3
   - then we calculate sigmoid:1/(1+e^(-3)) = 0.95
   so x is in class y=1 with the probability of 95%

4.

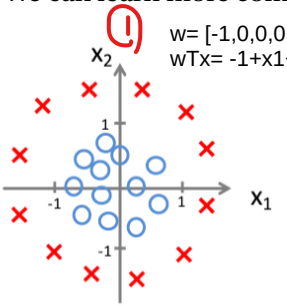


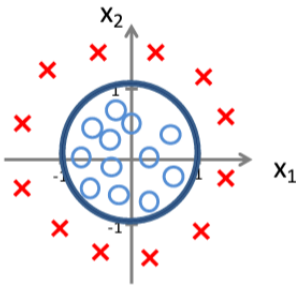
2. Predict $y = 1$ if $-3 + x_1 + x_2 \geq 0$
   y=0 o.w

Non-linear decision boundary example

$$\sigma(\mathbf{w}^T x) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2)$$

We can learn more complex decision boundaries when having higher order terms





2

Predict $y = 1$ if $-1 + x_1^2 + x_2^2 \geq 0$

y=0 o.w

now lets find w automatically

maximum liklihood estimation

in coin fliping:
in 16 times in total => 12 times came 1, 4 times 0

p : probability of 1 coming
1-p : probability of 0 coming

1 : 12 times
0 : 4 times

binomial distribution's probability function:

$$P(X=x \mid n) = \binom{n}{x} p^{x} (1-p)^{n-x}$$

x: Came, n: Total

$\binom{16}{12} p^{12} (1-p)^{4}$

probability of 12 out of 16 times , 1 coming :

now we want to find a p that probability of coming 1 , 12 times out of 16 times becomes max :

$$\arg\max_p \binom{16}{12} p^{12} (1-p)^{4} = \arg\max_p p^{12} (1-p)^{4}$$

log is an increasing function : so if log is max inside is max

$$\frac{\partial}{\partial p} 12 \log(p) + 4 \log(1-p) = \frac{12}{p} - \frac{4}{1-p} = 0 \Rightarrow p = \frac{12}{16}$$

we differentiate this in respect to p

now the probability of coming one in this coin is estimated :12/16

ML estimation

- We had posterior of a sample as:

$$P(y|\mathbf{x};\mathbf{w}) = \prod_i P(y^{(i)}|\mathbf{x}^{(i)};\mathbf{w})$$

x and y are independent from each other so we can multiply the probabilities

$$P(y^{(i)}|x^{(i)},\mathbf{w}) \begin{cases} P(y^{(i)}=1|\mathbf{x}^{(i)};\mathbf{w}) = \sigma(\mathbf{w}^T x) \\ P(y^{(i)}=0|\mathbf{x}^{(i)};\mathbf{w}) = 1-\sigma(\mathbf{w}^T\mathbf{x}) \end{cases}$$

- Logistic regression should maximize production of all these sample posteriors.
- Maximum (conditional) log likelihood:

$$\hat{\mathbf{w}} = \arg\max_{w} \log \prod_{i=1}^{n} P(y^{(i)}|x^{(i)},\mathbf{w})$$

- Note that in **binary** classification y is either 1 or 0, So we can have posterior term simplified as follows:

$$P(y^{(i)}|x^{(i)},\mathbf{w}) = \sigma(\mathbf{w}^T x^{(i)})^{y^{(i)}} (1-\sigma(\mathbf{w}^T x^{(i)}))^{(1-y^{(i)})}$$

ML estimation

- Logarithm of the posterior probability:

$$\log P(y^{(i)}|x^{(i)},\mathbf{w}) = y^{(i)}\log(\sigma(\mathbf{w}^T x^{(i)})) + (1-y^{(i)})\log(1-\sigma(\mathbf{w}^T x^{(i)}))$$

- Hence the log likelihood is as follows:

$$\log\prod_{i=1}^{n} P(y^{(i)}|x^{(i)},\mathbf{w}) = \sum_{i=1}^{n}\log P(y^{(i)}|x^{(i)},\mathbf{w}) \quad : \text{Cost function}$$

$$= \sum_{i=1}^{n}[y^{(i)}\log(\sigma(\mathbf{w}^T x^{(i)})) + (1-y^{(i)})\log(1-\sigma(\mathbf{w}^T x^{(i)}))] \quad \leftarrow \text{Simplify}$$

$$\begin{cases} P(y^{(i)}=1|x^{(i)};w) = (\sigma(w^T x))^{y^{(i)}} \\ P(y^{(i)}=0|x^{(i)};w) = (1-\sigma(w^T x))^{1-y^{(i)}} \end{cases}$$

$$\Rightarrow P(y^{(i)}|x^{(i)};w) = \sigma(w^T x)^{y^{(i)}}(1-\sigma(w^T x))^{1-y^{(i)}}$$

$y^{(i)}=1$ → stays if $y=1$; $y^{(i)}=0$ → stays if $y=1$

if we multiply them : we make two if statements into a single statement

Cost function

- We should find

$$\hat{\mathbf{w}} = \arg\min_{w} J(w)$$

- MLE finds parameters that best describe a classification problem so cost function should be negative of log likelihood term:

$$J(w) = -\sum_{i=1}^{n} \log P(y^{(i)}|x^{(i)}, \mathbf{w})$$

$$= \sum_{i=1}^{n} -y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1-y^{(i)}) \log(1-\sigma(\mathbf{w}^T x^{(i)}))$$

- No closed form solution for $\nabla_w J(w) = 0$
- However $J(w)$ is **convex**.

Cost function (cont.)

proof of convexity

- Convexity of $J(w)$ can easily be proved:
  - We use the lemma that sum of several convex functions is still convex (you can prove it on your own).
  - Each term in the summation is differentiable (twice).
  - If you twice get derivative of (with respect to σ):

$$-y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))$$

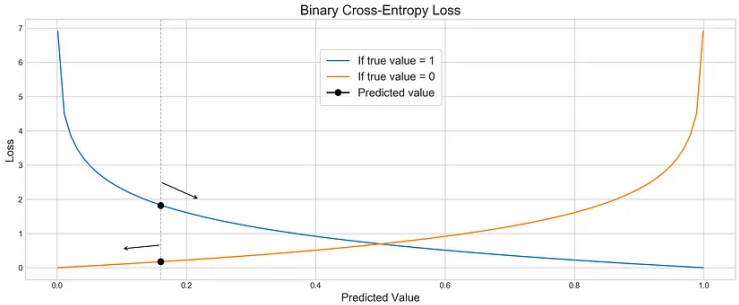
  - You get:

$$\frac{y}{\sigma^2} + \frac{1-y}{(1-\sigma)^2}$$

  - Which for both $y = 0$ and $y = 1$ is positive.
  - Each $\log P(y^{(i)} | x^{(i)}, \mathbf{w})$ is convex, hence the summation is convex as well.

Cost function (cont.)

- Visualization of each binary cross entropy loss term:



- As you can see if the model predicted value is $\hat{y} = 0.16$ and true label is $y = 1$ then the error is high but if the true label is $y = 0$ the error would be low.

Figure adopted from https://towardsdatascience.com/logistic-regression-from-scratch-69db4f587e17

Gradient descent

- Remember from previous slides:

$$J(w) = \sum_{i=1}^{n} -y^{(i)} \log(\sigma(\mathbf{w}^T x^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T x^{(i)}))$$

- Update rule for **gradient descent**:

$$w^{t+1} = w^t - \eta \nabla_w J(w^t)$$

- With $J(w)$ definition for logistic regression we get:

$$\nabla_w J(w) = \sum_{i=1}^{n} (\sigma(\mathbf{w}^T x^{(i)}) - y^{(i)}) x^{(i)}$$

Gradient descent

- Compare the gradient of logistic regression with the gradient of SSE in linear regression :

$$\nabla_w J(w) = \sum_{i=1}^{n} (\sigma(\mathbf{w}^T x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\nabla_w J(w) = \sum_{i=1}^{n} (\mathbf{w}^T x^{(i)} - y^{(i)}) x^{(i)}$$

Loss function

- Loss function is a single overall measure of loss incurred for taking our decisions (over entire dataset).
- We have:

$$Loss(y, \sigma(\mathbf{w}^T x)) = -y \times \log(\sigma(\mathbf{w}^T x)) - (1-y) \times \log(1-\sigma(\mathbf{w}^T x))$$

- Since in binary classification either $y=1$ or $y=0$ we have:

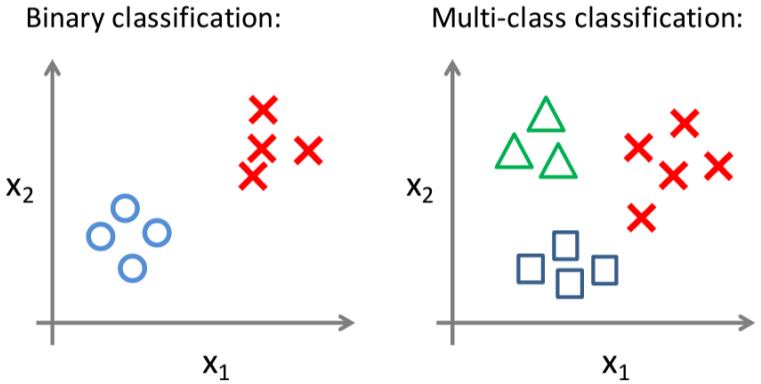
$$Loss(y, \sigma(\mathbf{w}^T x)) = \begin{cases} -\log(\sigma(\mathbf{w}^T x)) & \textbf{if } y=1 \\ -\log(1-\sigma(\mathbf{w}^T x)) & \textbf{if } y=0 \end{cases}$$

- How is it related to zero-one loss? ($\hat{y}$ is the predicted label and y is the ture label)

$$Loss(y, \hat{y}) = \begin{cases} 1 & \textbf{if } y \neq \hat{y} \\ 0 & \textbf{if } y = \hat{y} \end{cases}$$

Multi-class logistic regression

- Now consider a problem where we have K classes and every sample only belongs to one class (for simplicity).



Multi-class logistic regression (cont.)

- For each class k, $\sigma_k(x;\mathbf{W})$ predicts the probability of $y = k$.
  - i.e., $P(y = k|x, \mathbf{W})$
- For each data point x_0, $\sum_{k=1}^{K} P(y = k|x_0, \mathbf{W})$ must be 1
  - W denotes a matrix of w_i's in which each w_i is a weight vector dedicated for class label i.
- On a new input x, to make a prediction, we pick the class that maximizes $\sigma_k(x;\mathbf{W})$:

$$\alpha(x) = \arg\max_{k=1,\ldots,K} \sigma_k(x;\mathbf{W})$$

if $\sigma_k(x;\mathbf{W}) > \sigma_j(x;\mathbf{W})\ \forall j \neq k$ **then decide** C_k

Multi-class logistic regression (cont.)

- $K > 2$ and $y \in \{1, 2, \ldots, K\}$

$$\sigma_k(x, \mathbf{W}) = P(y = k|x) = \frac{\exp(w_k^T x)}{\sum_{j=1}^{K} \exp(w_j^T x)}$$

- Normalized exponential (Aka **Softmax**)
- if $w_k^T x \gg w_j^T x$ for all $j \neq k$ then $P(C_k|x) \approx 1$ and $P(C_j|x) \approx 0$
- Note : remember from Bayes theorem:

$$P(C_k|x) = \frac{P(x|C_k)P(C_k)}{\sum_{j=1}^{K} P(x|C_j)P(C_j)}$$

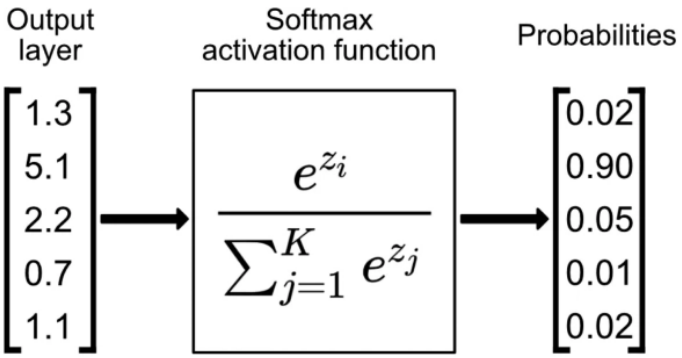
Multi-class logistic regression (cont.)

- Softmax function **smoothly** highlights the maximum probability and is differentiable.
- Compare it with max(.) function which is strict and non-differentiable
- Softmax can also handle negative values because we are using exponential function
- And it gives us probability for each class since:

$$\sum_{k=1}^{K} \frac{\exp(w_k^T x)}{\sum_{j=1}^{K} \exp(w_j^T x)} = 1$$

Multi-class logistic regression (cont.)

- An example of applying softmax (note that $z_i = w^T x_i$):



Multi-class logistic regression (cont.)

- Again we set $J(W)$ as negative of log likelihood.
- We need $\hat{W} = \arg\min_W J(W)$

$$
\begin{aligned}
J(W) &= -\log \prod_{i=1}^{n} P(y^{(i)} | x^{(i)}, \mathbf{W}) \\
&= -\log \prod_{i=1}^{n} \prod_{k=1}^{K} \sigma_k(x^{(i)}; \mathbf{W})^{y_k^{(i)}} \\
&= -\sum_{i=1}^{n} \sum_{k=1}^{K} y_k^{(i)} \log(\sigma_k(x^{(i)}; \mathbf{W}))
\end{aligned}
$$

- If **i-th** sample belongs to class k then $y_k^{(i)}$ is 1 else 0.
- Again no closed-from solution for $\hat{W}$

Multi-class logistic regression (cont.)

- From previous slides we have:

$$J(W) = -\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)} \log(\sigma_k(x^{(i)}; \mathbf{W}))$$

- In which:

$$W = [w_1, w_2, \ldots, w_K], \quad Y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & \cdots & y_K^{(1)} \\ y_1^{(2)} & \cdots & y_K^{(2)} \\ \vdots & \ddots & \vdots \\ y_1^{(n)} & \cdots & y_K^{(n)} \end{pmatrix}$$

- y is a vector of length K (1-of-K encoding)
  - For example $y = [0, 0, 1, 0]^T$ when the target class is C_3.

Multi-class logistic regression (cont.)

- Update rule for gradient descent:

$$w_j^{t+1} = w_j^t - \eta \nabla_W J(W^t)$$

$$\nabla_{w_j} J(W) = \sum_{i=1}^{n} (\sigma_j(x^{(i)}; \mathbf{W}) - y_j^{(i)}) x^{(i)}$$

- w_j^t denotes the weight vector for class j (since in multi-class LR, each class has its own weight vector) in the t-th iteration

Logistic regression (LR) summary

- LR is a **linear** classifier
- LR optimization problem is obtained by **maximum likelihood**
- No closed-form solution for its optimization problem
  - But convex cost function and global optimum can be found by gradient ascent

Probabilistic view in classification problem

- In a classification problem:
  - Each **feature** is a **random variable** (e.g. a person's height)
  - The **class label** is also considered a **random variable** (e.g. a person could be overweight or not)
- We observe the feature values for a random sample and intend to find its class label
  - Evidence: Feature vector x
  - Objective: Class label

Definitions

- Posterior probability : The probability of a class label C_k given a sample x

$$P(C_k|x)$$

- Likelihood or class conditional probability : PDF of feature vector x for samples of class C_k

$$P(x|C_k)$$

- Prior probability : Probability of the label be C_k

$$P(C_k)$$

- $P(x)$: PDF of feature vector x
  - From total probability theorem:

$$P(x) = \sum_{k=1}^{K} P(x|C_k)P(C_k)$$

Probabilistic classifiers

- Probabilistic approaches can be divided in two main categories:
  - Generative
    - Estimate PDF $P(x, C_k)$ for each class C_k and then use it to find $P(C_k|x)$. Alternatively estimate both PDF $P(x|C_k)$ and $P(C_k)$ to find $P(C_k|x)$.
  - Discriminative
    - Directly estimate $P(C_k|x)$ for class C_k

Probabilistic classifiers (cont.)

- Let's assume we have input data x and want to classify the data into labels y.
- A generative model learns the **joint** probability distribution $P(x, y)$.
- A discriminative model learns the **conditional** probability distribution $P(y|x)$

Discriminative vs. Generative : example

- Suppose we have the following dataset in form of (x, y):

$$(1,0),(1,0),(2,0),(2,1)$$

- $P(x, y)$ is :

	$y=0$	$y=1$
$x=1$	$\frac{1}{2}$	0
$x=2$	$\frac{1}{4}$	$\frac{1}{4}$

- $P(y|x)$ is :

	$y=0$	$y=1$
$x=1$	1	0
$x=2$	$\frac{1}{2}$	$\frac{1}{2}$

Discriminative vs. Generative : example (cont.)

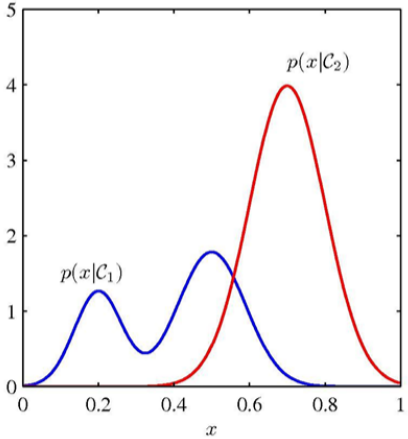
- The distribution $P(y|x)$ is the natural distribution for classifying a given sample x into class y.
  - This is why that algorithms which model this directly are called **discriminative** algorithms.
- Generative algorithms model $P(x, y)$, which can be transformed into $P(y|x)$ by Bayes rule and then used for classification.
  - However, the distribution $P(x, y)$ can also be used for other purposes.
  - For example we can use $P(x, y)$ to **generate** likely (x, y) pairs

Generative approach

1. Inference
   - Determine class conditional densities $P(x|C_k)$ and priors $P(C_k)$
   - Use Bayes theorem to find $P(C_k|x)$
2. Decision
   - Make optimal assignment for new input (after learning the model in the inference stage)
   - if $P(C_i|x) > P(C_j|x) \forall j \neq i$, then decide C_i .

Generative approach (cont.)

- Generative approach for a binary classification problem:



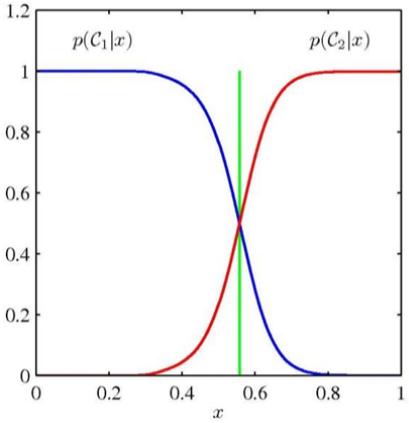
Figures adapted from Machine Learning and Pattern Recognition, Bishop

Discriminative approach

1. Inference
   - Determine the posterior class probabilities $P(C_k|x)$ directly.
2. Decision
   - Make optimal assignment for new input (after learning the model in the inference stage)
   - if $P(C_i|x) > P(C_j|x) \forall j \neq i$, then decide C_i .

Discriminative approach (cont.)

- Discriminative approach for a binary classification problem:



Figures adapted from Machine Learning and Pattern Recognition, Bishop

Discriminative approach (cont.)

- Logistic regression is a **discriminative** approach.
- We directly want to specify the class label with $\sigma(\mathbf{w}^T x)$

Contributions

- **These slides are authored by:**
  - Danial Gharib

[1] M. Soleymani Baghshah, “Machine learning.” Lecture slides.

[2] A. Ng, “Ml-005, lecture 6.” Lecture slides.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*.
Information Science and Statistics, New York, NY: Springer, 1 ed., Aug. 2006.

[4] S. Fidler, “Csc411.” Lecture slides.

[5] A. Ng and T. Ma, *CS229 Lecture Notes*.
Updated June 11, 2023.

Any Questions?